CNSCC.405, Lab 6

Simulation of the Performance of OFDM Systems: OFDM 系统性能仿真：

Tasks: 1.

The Chapter from the book is attached to perform this Lab:

Narasimhamurthy, A.B., Banavar, M.K., Tepedelenlioğlu, C. (2010). Simulation of the Performance of OFDM Systems. In: OFDM Systems for Wireless Communications. Synthesis Lectures on Algorithms and Software in Engineering. Springer, Cham. https://doi.org/10.1007/978-3-031-01513-7_6

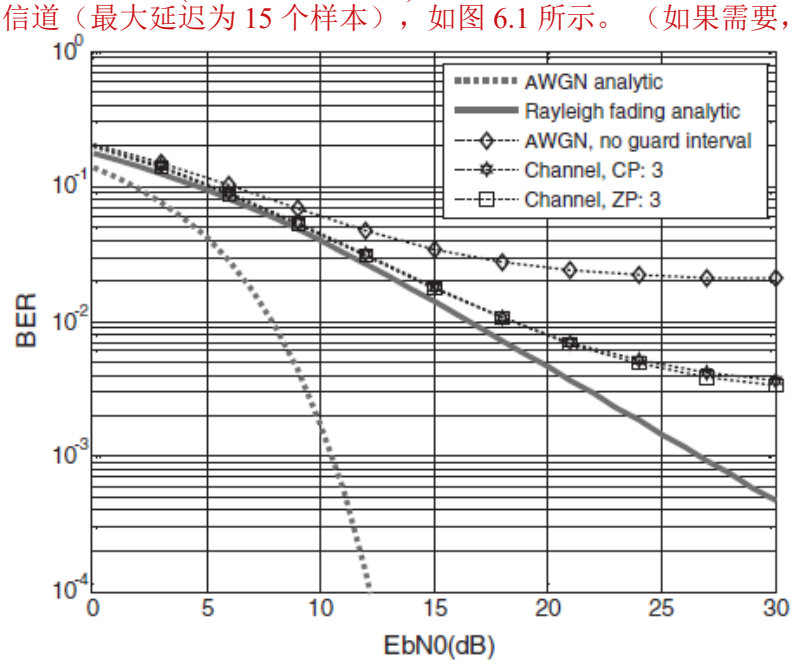
a. Repeat the different tasks provided in the attached chapter. 1. 重复所附章节中提供的不同任务

b. Explore every code and operation provided. 2. 探索提供的每个代码和操作

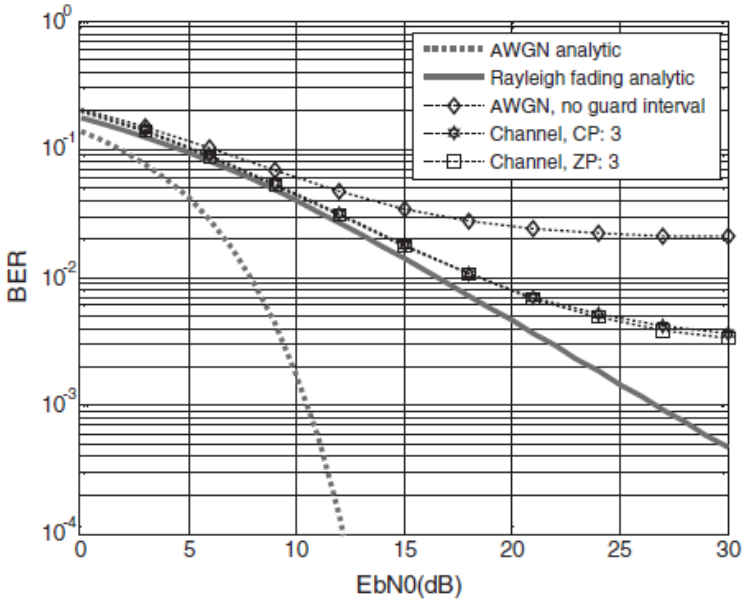
Tasks: 2.

With the information and knowledge from Task 1 and Lectures week 3 and week 4, you are required to simulate the effect of ISI as the length of a guard interval (CP, CS, or ZP) varies. Considers the BER performance of an OFDM system with 64-point FFT (N= 64) and 16 virtual carriers (N_{vc}=N-N_{sub} = 16), for 16-QAM signaling in the AWGN or a multipath Rayleigh fading channel (with the maximum delay of 15 samples) as shown in Figure 6.1. (Make any realistic assumptions if required)

借助任务 1 以及讲座第 3 周和第 4 周的信息和知识，您需要模拟保护间隔（CP、CS 或 ZP）长度变化时 ISI 的影响。考虑具有 64 点 FFT (N= 64) 和 16 个虚拟载波 (Nvc=N-Nsub = 16) 的 OFDM 系统的 BER 性能，适用于 AWGN 中的 16-QAM 信令或多径瑞利衰落信道（最大延迟为 15 个样本），如图 6.1 所示。（如果需要，做出任何现实的假设）



(b) GI length: $N_g = 3$



(b) GI length: $N_g = 3$

Figure 6.1: BER performance for OFDM system with 16-QAM.

Tasks: 3.

Write a summary of what you have learned, observed, things went wrong (if in your experiment, you have not had the predicted result), and how you can overcome the similar problem in future.

写下你所学到的、观察到的、出现问题的总结（如果在你的实验中，你没有得到预测的结果），以及你将来如何克服类似的问题。

CHAPTER 6

Simulation of the Performance of OFDM Systems

6.1 PERFORMANCE OF AN OFDM SYSTEM

In the performance analysis of an OFDM system, we assume that the channel remains constant for a certain length of time and then randomly changes to an independent value. This behavior is termed as quasi-static fading. Assuming quasi-static channel and perfect synchronization at the receiver leads to the received signals on the various subcarriers (SC) to be independent of each other. Therefore, the channel on each SC can be equivalently represented as a flat fading channel with additive white Gaussian noise (AWGN). The instantaneous signal to noise ratio (SNR) on each subcarrier, within a block of quasi-static fading, can now be represented as

$$\gamma_k = \bar{\gamma}|H[k]|^2, \tag{6.1}$$

where $\bar{\gamma}$ represents the average SNR on the subcarrier and $H[k]$ represents the channel on the kth subcarrier as given in (3.12). Note here that we have assumed that all SC's have the same average SNR $\bar{\gamma} = \mathrm{E_b}/N_o$, where $\mathrm{E_b}$ represents the average energy per bit and N_o represents the height of the noise spectral density expressed in the units of Watts per Hertz.

Further, given that the subcarriers are i.i.d., and assuming perfect channel knowledge at the receiver, the probability of error, $\mathrm{P_e}$, of the OFDM system can be expressed as the mean of the probability of error of individual subcarriers, i.e.,

$$\mathrm{P_e} = \frac{1}{N}\sum_{k=1}^{N}\mathrm{P_e}[k], \tag{6.2}$$

where $\mathrm{P_e}[k]$ is the channel dependent instantaneous probability of error on the kth subcarrier. $\mathrm{P_e}[k]$ depends on the modulation scheme chosen. For binary phase shift keying (BPSK) modulation [65, Chap. 3], we have

$$\mathrm{P_e}[k] = \mathrm{Q}\left(\sqrt{2\gamma_k}\right). \tag{6.3}$$

If instead, quadrature phase shift keying (QPSK) modulation is used, we have

$$\mathrm{P_e}[k] = 2\mathrm{Q}\left(\sqrt{\gamma_k}\right) - \mathrm{Q}^2\left(\sqrt{\gamma_k}\right). \tag{6.4}$$

For other modulation schemes, instantaneous probability of error expressions can be found in [19].

A. B. Narasimhamurthy et al., *OFDM Systems for Wireless Communications*
© Springer Nature Switzerland AG 2010

To average the probability of error over time, the instantaneous value has to be averaged across all possible values of the random variable $H[k]$, i.e.,

$$\overline{\mathrm{P}}_{\mathrm{e}}[k] = \mathrm{E}_H\left[\mathrm{P}_{\mathrm{e}}[k]\right], \tag{6.5}$$

where $\mathrm{E}_H[\cdot]$ denotes the expectation operator with respect to $H[k]$. Therefore, using (6.5), the average probability of error of a OFDM system can be expressed as

$$\overline{\mathrm{P}}_{\mathrm{e}} = \frac{1}{N}\sum_{k=1}^{N}\mathrm{E}_H\left[\mathrm{P}_{\mathrm{e}}[k]\right]. \tag{6.6}$$

At the receiver, a maximum likelihood (ML) decoder is implemented. An ML decoder, as the name implies, maximizes the likelihood of receiving a signal, $\mathbf{y}$, conditioned on the transmitted signal, $\mathbf{s}$, and channel, $\mathbf{H}$, i.e.,

$$\hat{\mathbf{s}} = \underset{\mathbf{s}}{\operatorname{argmin}}\, p(\mathbf{y}|\mathbf{s}, \mathbf{H}) = \underset{\{s_k\}}{\operatorname{argmax}} \prod_{k=1}^{N} p(y_k|s_k, h_k). \tag{6.7}$$

For all simulations considered, we assume Rayleigh fading channels and additive white Gaussian noise (AWGN), thereby leading to an ML decoder of the form [19],

$$\hat{\mathbf{s}} = \underset{\mathbf{s}}{\operatorname{argmax}}\, \|\mathbf{y} - \mathbf{H}\mathbf{s}\|^2. \tag{6.8}$$

For the case when frequency and timing synchronization is perfect at the receiver, ML decoder can be expressed as

$$\hat{s}[k] = \underset{s[k]}{\operatorname{argmin}}\, |y[k] - H[k]s[k]|^2. \tag{6.9}$$

Note here that the ML decoder can be represented as in (6.9) because there is no channel coding. Instead, if channel coding was employed then sequence detection algorithms like the Viterbi algorithm [17] needs be used.

In the following section, we implement the Monte-Carlo method to calculate the average probability of error of an OFDM system. The Monte-Carlo method is a numerical method to estimate the ensemble average with respect to a random variable. Further information on how this technique can be used to approximate the expected value can be found in [66, 67, 68, 69, 70].

6.2 SIMULATIONS

In what follows, MATLAB simulations are used to demonstrate the working of OFDM, based on the theoretical development of OFDM has been presented previously.

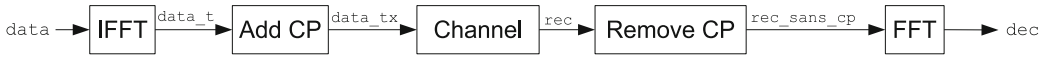


Figure 6.1: Basic OFDM model.

6.2.1 THE BASIC OFDM SYSTEM

In this section, the basic OFDM system is simulated. As discussed in the preceding chapters, a basic OFDM system is constructed as shown in Figure 6.1. In this section, MATLAB code will be provided for each of the blocks, with an explanation. The full program for the basic OFDM system is then provided, including flexibility to vary several parameters and visualization options.

Data Generation and Modulation

Data is first generated to be transmitted over the OFDM system. Data of length `N` is randomly generated and modulated as shown:

```
am = [-1,1];
M = 2;
dat_ind = ceil(M*rand(1,N));
data = am(dat_ind);
```

These lines of code generate a baseband representation of BPSK signals (± 1). The data is generated from a source that generates the symbols with equal probability. It should be noted here that BPSK is not the only mode of modulation, and any other modulation scheme can be used at this stage. For example, if QPSK is used, the code can be modified as follows:

```
am = [1,1i,-1,-1i];
M = 4;
dat_ind = ceil(M*rand(1,N));
data = am(dat_ind);
```

IFFT

The first operation performed on the data is a `N`-point IFFT. In MATLAB, the IFFT function can be used for this operation as follows:

```
data_t = ifft(data);
```

Add Cyclic Prefix

After the IFFT, the last few data-points are repeated at the beginning. The repeated data is called the cyclic prefix.

```
data_cp = [data_t(end-CP_length+1:end), data_t];
```

Cyclic prefix of length `CP_length` is added to the beginning of the data block.

Channel

Transmission occurs over frequency selective fading channels. The channels are modeled as FIR filters of order L. The channel, h, is assumed to have channel taps drawn from a Rayleigh distribution. The power per tap is normalized, and the channel can be simulated as

```
h = complex(randn(L+1,1), randn(L+1,1))*sqrt(0.5/(L+1));
```

Noise is generated to be added to the transmission over the channel. The noise is zero-mean complex Gaussian. The code below shows how to generate the noise:

```
noise = complex(randn(1,Total_length), randn(1,Total_length))*sqrt(0.5/N);
```

Since the channel is modeled as an FIR filter, the output of the channel is computed by filtering the input signal with the channel, and noise is then added on to it. To adjust the SNR, the transmission is scaled by a power value as shown:

```
rho = SNR;
rec = sqrt(rho)*(filter(h,1,data_cp))+noise
```

Remove Cyclic Prefix

The cyclic prefix is removed from the received data. The first `CP_length` symbols are discarded from the received data:

```
rec_sans_cp = rec(CP_length+1:end)
```

FFT and Demodulation

The data extracted by discarding the cyclic prefix is transformed into the frequency domain. The data in the frequency domain is then equalized to account for the channel, and to yield the final received signal. It should be noted here that the FFT used here to convert the channel into the frequency domain is normalized by the number of channel taps. The received signal is then demodulated to obtain an estimate of the transmitted signals. In the following code, it is assumed that BPSK modulation has been used at the transmitter:

```
rec_f = fft(rec_sans_cp); % FFT
h_f = sqrt(rho)*fft(h,N); % Equivalent channel on each subcarrier
det1 = abs(rec_f+h_f).^2; % Calc the Euclidean dist
                          % assuming -1 was transmitted
det2 = abs(rec_f-h_f).^2; % Calc the Euclidean dist
                          % assuming +1 was transmitted
det = [det1, det2];       % Concatenating the two vectors

% Find the symbol the received signal is closest to
[min_val, ind] = min(det, [], 2);
```

```
dec = 2*((ind-1)>0.5)-1;  % BPSK decoding
```

For QPSK decoding:

```
rec_f = fft(rec_sans_cp); % FFT

% Equivalent channel on each subcarrier
h_f = sqrt(rho)*fft(h,N);

% Calc the Euclidean dist assuming 1 was transmitted
det1 = abs(rec_f-am(1)*h_f).^2;

% Calc the Euclidean dist assuming +1i was transmitted
det2 = abs(rec_f-am(2)*h_f).^2;

% Calc the Euclidean dist assuming -1 was transmitted
det3 = abs(rec_f-am(3)*h_f).^2;

% Calc the Euclidean dist assuming -1i was transmitted
det4 = abs(rec_f-am(4)*h_f).^2;

% Concatenating the vectors
det = [det1, det2, det3, det4];

% Find the symbol the received signal is closest to
[min_val, ind] = min(det, [], 2);

% Generating the decoded symbols
dec = am(ind);
```

The received symbols are decoded using maximum likelihood (ML) estimation.

6.2.1.1 OFDM Simulation

A Monte-Carlo simulation is performed to estimate the probability of error of the received signals when transmission occurs using OFDM. The SNR of the channels is varied and the probability of error is estimated for each case of SNR. The program for the full simulation model is provided. In the code provided, it is assumed that data is transmitted in frames, and each frame consists of `B` OFDM symbols. This allows us to evaluate the effects of inter-block-interference when the length of the cyclic prefix is less than the number of channel taps. The simulation results are presented for different parameters, which can be changed within the code.

```
clear all
clc

N = 16;            % Number of subcarriers in each OFDM symbol
L = 3;             % Channel order
CP_length = 4;     % Cyclic prefix length
B = 10;            % Number of OFDM symbols per transmitted frame
mc_N = 5000;       % Number of iterations to achieve sufficient errors
SNR_db = 0:2:20;                    %SNR in dB
SNR = 10.^(SNR_db/10);              % SNR values
Pe = zeros(size(SNR_db));           % Initializing the error vector
Total_length = (CP_length+N)*B;     % Total length of each frame
am = [-1,1];       % For BPSK
M = 2;             % For BPSK

for SNR_loop = 1:length(SNR_db)
    rho = SNR(SNR_loop);
    err = 0;

    for mc_loop = 1:mc_N
        dat_ind = ceil(M*rand(B,N));
        data = am(dat_ind);

        % Reshaping the data into a BxN matrix,...
        %...used later for error detection
        data_reshape = reshape(data, 1, B*N);

        tx_data = data;

        for b = 1:B
            % Taking the IFFT
            data_t(b,:) = ifft(tx_data(b,:));
        end

        % Adding Cyclic prefix
        data_cp = [data_t(:,end-CP_length+1:end), data_t];

        % Reshape the BxN matrix to obtain the frame (1xTotal_length)
        data_tx =reshape(data_cp.',1,Total_length);
```

```
h = complex(randn(L+1,1), randn(L+1,1))*sqrt(0.5/(L+1));

%Noise
noise = complex(randn(1,Total_length), ...
                   randn(1,Total_length))  * sqrt(0.5/N);
% Received signal
rec = sqrt(rho)*(filter(h,1,data_tx))+noise;

% Reshape the recd signal into CP_length+N x B array
rec_reshaped = (reshape(rec, CP_length+N, B)).';

% Remove CP
rec_sans_cp = rec_reshaped(:,CP_length+1:end);

for bb = 1:B
    % Taking the FFT
    rec_f(bb,:) = fft(rec_sans_cp(bb,:));
end

% Calculating the equivalent channel on each subcarrier
h_f = sqrt(rho)*fft(h,N);

for b2 = 1:B

    % Extracting the OFDM symbol from the "rec_f" matrix
    rec_symbol = transpose(rec_f(b2,:));

    % Calc Euclidean dist assuming -1
    det1 = abs(rec_symbol+h_f).^2;

    % Calc Euclidean dist assuming +1
    det2 = abs(rec_symbol-h_f).^2;

    % Concatenate the two vectors
    det = [det1, det2];

    % Find symbol the recd signal is closest to
    [min_val, ind] = min(det, [], 2);
```

```
            % Generate the decoded symbols
            dec(b2,:) = 2*((ind-1)>0.5)-1;
        end

        % Reshape the decoded symbols to calc error
        dec_reshape = reshape(dec, 1, B*N);

        % Comparing dec_reshape against...
        %...data_reshape to calculate errors
        err = err + sum(dec_reshape~=data_reshape);
    end
    % Calculate the probability of error
    Pe(SNR_loop) = err/(mc_N*B*N);
end

% Semilog plot of Pe vs. SNR_db
semilogy(SNR_db,Pe)
```

Figure 6.2 shows the output of the code for two modulation schemes, BPSK and QPSK. It is shown that the performance of the system remains the same irrespective of the number of subcarriers, as long as the number of symbols in the cyclic prefix is at least as many as the number of channel taps. Furthermore, as expected from digital communications, the symbol error rate of the QPSK system is worse than the BPSK case. In Figure 6.3, the system is simulated for QPSK modulation. Three curves are plotted, each for a different length of cyclic prefix. It can be seen that as the length of the cyclic prefix reduces, inter-block-interference (IBI) causes deterioration in performance.

6.2.2 CARRIER FREQUENCY OFFSET 载波频率偏移

Carrier frequency offset and its drawbacks were discussed in Chapter 4. In this Section, we will provide simple code to simulate the effect of frequency offset and demonstrate its effect on the BER performance of the OFDM system. More detailed treatment can be found in [71, 72].

6.2.2.1 Simulation

The frequency offset is provided as a complex exponential multiplier for each subcarrier. This simulates the effect of having the frequency of each subcarrier being offset at the receiver by a small amount. In the code that follows, the frequency offset is varied, and the BER is calculated for each of those values, to demonstrate the effect of CFO on performance of OFDM systems. Similar to the previous cade, the code is flexible and different parameters such as modulating techniques, levels of offset and number of Monte-Carlo iterations can be changed.

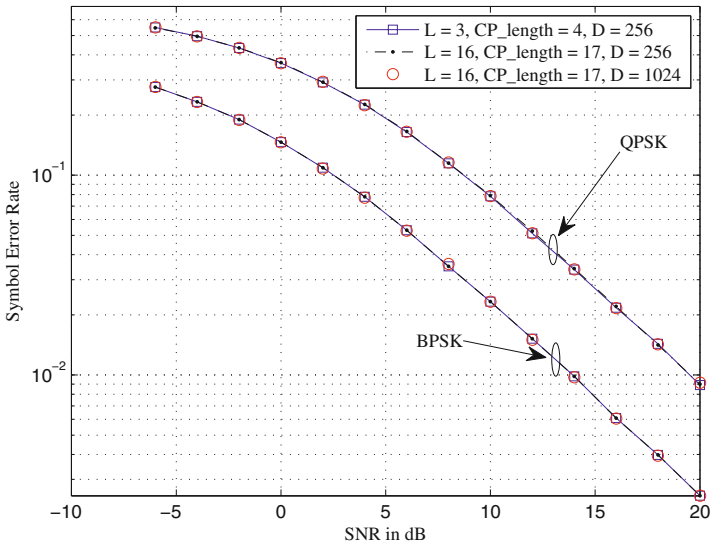


Figure 6.2: Simulation of the basic OFDM model. Plot shows effect of SNR on the probability of error.

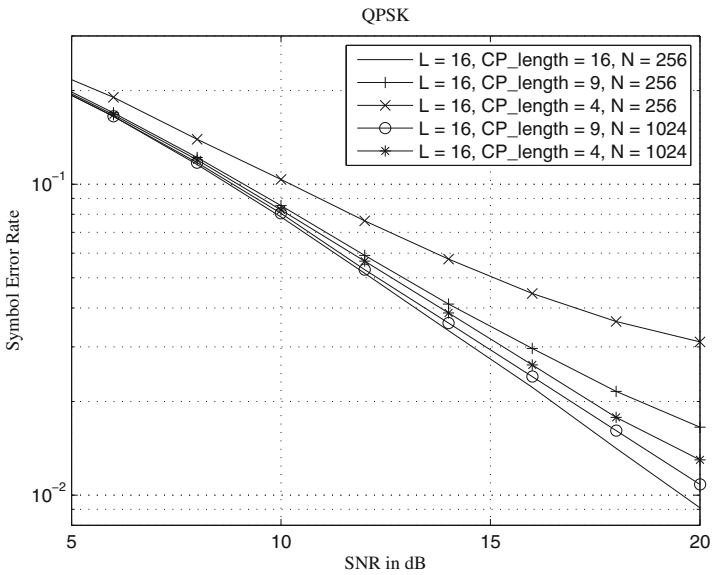


Figure 6.3: Simulation of the basic OFDM model, showing the need for cyclic prefix.

```
clear all
clc

N = 16;         % Number of subcarriers in each OFDM symbol
L = 3;          % Channel order
CP_length = 4; % Cyclic prefix length
B = 1;          % Number of OFDM symbols per transmitted frame
mc_N = 5000;    % Number of iterations to achieve sufficient errors
SNR_db = 5;     % SNR in dB

SNR = 10.^(SNR_db/10); % SNR

Pe = zeros(size(SNR_db));

Total_length = (CP_length+N)*B; % Total length of each frame
am = [-1,1];    % For BPSK
M = 2;          % For BPSK

freq_offset = [-0.5:0.01:0.5];

for off_loop = 1:length(freq_offset)
    rho = SNR;
    err = 0;
    for mc_loop = 1:mc_N

        dat_ind = ceil(M*rand(B,N));
        data = am(dat_ind);

        % Reshaping the data into a BxN matrix,...
        %...used later for error detection
        data_reshape = reshape(data, 1, B*N);

        tx_data = data;

        for b = 1:B
            % Taking the IFFT
            data_t(b,:) = ifft(tx_data(b,:));
        end
```

```
% Adding Cyclic prefix
data_cp = [data_t(:,end-CP_length+1:end), data_t];

% Reshape the BxN matrix to...
%...obtain the frame (1xTotal_length)
data_tx =reshape(data_cp.',1,Total_length);
h = complex(randn(L+1,1), randn(L+1,1))*sqrt(0.5/(L+1));
noise = complex(randn(1,Total_length), ...
                   randn(1,Total_length)) *sqrt(0.5/N); %Noise
rec = sqrt(rho)*(filter(h,1,data_tx))...
               * exp(-1i*2*pi*freq_offset(off_loop))+noise;

% Reshape the recd signal...
%...into CP_length+N x B array
rec_reshaped = (reshape(rec, CP_length+N, B)).';

% Remove CP
rec_sans_cp = rec_reshaped(:,CP_length+1:end);

for bb = 1:B

    % Taking the FFT
    rec_f(bb,:) = fft(rec_sans_cp(bb,:));
end

% Calculating the equivalent channel on each subcarrier
h_f = sqrt(rho)*fft(h,N);

for b2 = 1:B

    % Extracting the OFDM symbol from...
    %...the "rec_f" matrix
    rec_symbol = transpose(rec_f(b2,:));

    % Calc Euclidean dist assuming -1
    det1 = abs(rec_symbol+h_f).^2;

    % Calc Euclidean dist assuming +1
    det2 = abs(rec_symbol-h_f).^2;
```

```
            % Concatenate the two vectors
            det = [det1, det2];

            % Find symbol the recd signal is closest to
            [min_val, ind] = min(det, [], 2);

            % Generate the decoded symbols
            dec(b2,:) = 2*((ind-1)>0.5)-1;
        end

        % Reshape the decoded symbols to calc error
        dec_reshape = reshape(dec, 1, B*N);

        % Compare dec_reshape against...
        %...data_reshape to calc errors
        err = err + sum(dec_reshape~=data_reshape);
    end

    % Calculating the probability of error
    Pe(off_loop) = err/(mc_N*B*N)
end

%Semilog plot of Pe vs. offset
semilogy(freq_offset,Pe)
```

The code provided yields a value for the probability of error at each value of frequency offset. The value of probability of error is plotted for each value of offset. As expected, the best performance is obtained for a zero offset. As the offset increases in either direction, the performance deteriorates, and the performance is symmetric in the offset about the zero offset point. This plot for a channel SNR of 5dB is shown in Figure 6.4.

6.2.3 PAPR SIMULATIONS

The effect of high peak-to-average-power ratio (PAPR) on the performance of OFDM systems was seen in Chapter 4. In this simulation, the transmitted signal is simulated to provide an example of the transmit power over a few time samples and to demonstrate the occurrence of high PAPR. In this simulation, the base-band equivalent model of an OFDM system is considered. At each time instant, data is BPSK modulated and transmitted using OFDM. The instantaneous peak transmit power and the average transmit power are calculated and plotted in Figure 6.5. The number of subcarriers used is 16. It can be verified from (5.3) that the average power of the system must be

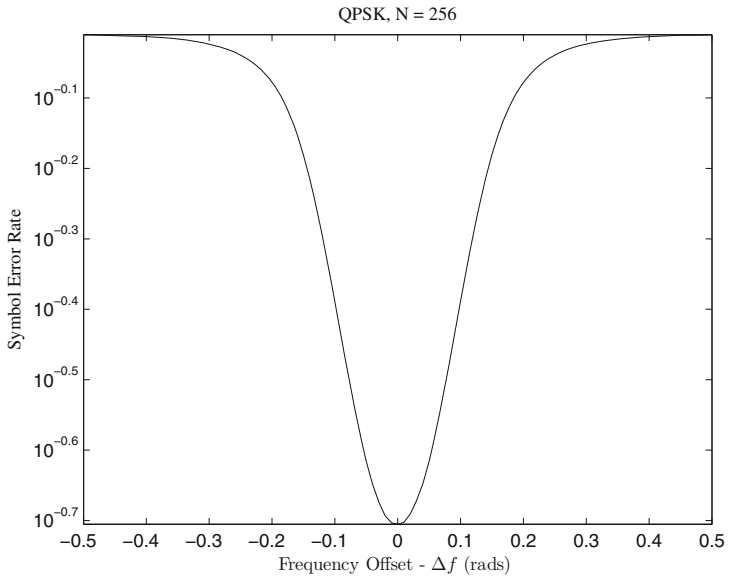


Figure 6.4: Bit error rate vs. phase offset

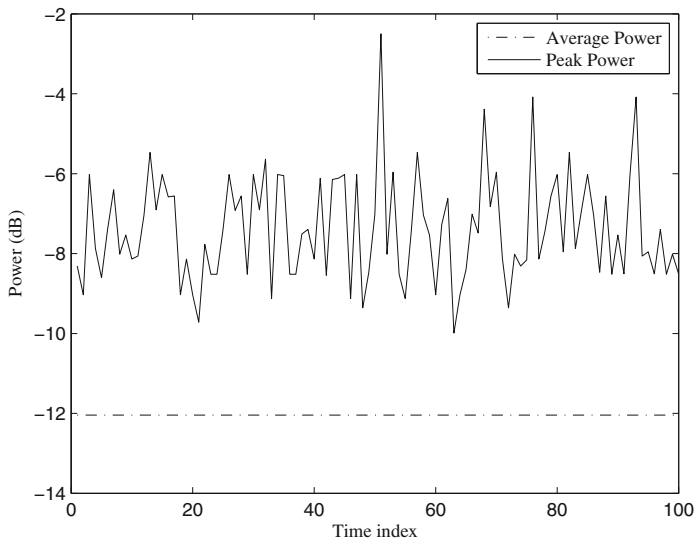


Figure 6.5: Peak power due to addition of in-phase sinusoids, compared with the average power of the transmissions.

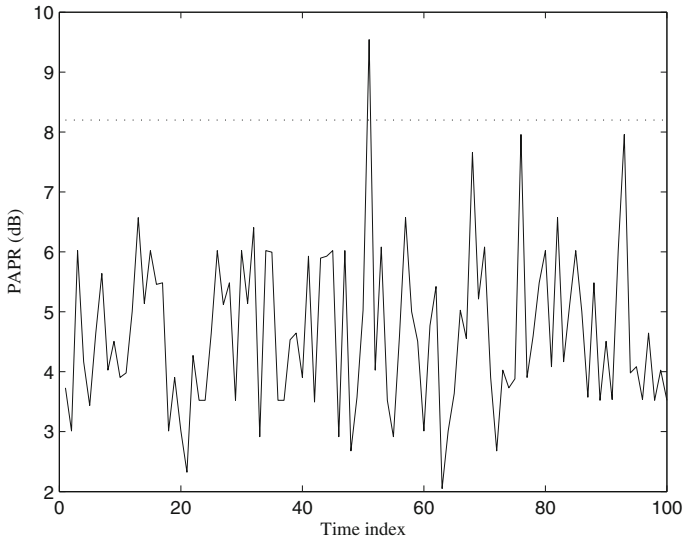


Figure 6.6: Peak to average power ratio compared against a threshold.

-12dB as can be seen from the figure. The PAPR is also calculated and plotted in Figure 6.6. As can be seen from (5.4), the maximum possible value of PAPR is 12dB, and in the case of this simulation, the system does not exceed this value. In fact, the largest PAPR in this case approaches 10dB. A predetermined threshold is also shown, and the PAPR exceeds this threshold at one point.

```
clear all

N = 16; % Length of data
SNR_db = 5;
SNR = 10.^(SNR_db/10);
Pe = zeros(size(SNR_db));
noise_var = 1/SNR;
time_samples = 100;
avg_pow = zeros(1,time_samples);
mx_pow = zeros(1,time_samples);
papr = zeros(1,time_samples);

for time_loop = 1:time_samples
    data = 2*(randn(N,1)>0)-1;
    data_t = ifft(fftshift(data));
```

```
    avg_pow(time_loop) = (norm(data_t))^2/N;
    mx_pow(time_loop) = max(data_t.*conj(data_t));
    papr(time_loop) = mx_pow(time_loop)/avg_pow(time_loop);
end

figure(1)
plot(10*log10(avg_pow))
hold all
plot(10*log10(mx_pow))
figure(2)
plot(10*log10(papr))
```

One of the ways of limiting PAPR at the transmitter is by clipping signals that exceed a certain level. In the code provided, an OFDM system is considered with such a clipping systems. Different clipping thresholds are considered to show the effect of clipping level on performance. In this example, the system considered has no additive noise, and the channel is frequency flat with gain one.

```
clear all
clc

N = 16;           % Number of subcarriers in each OFDM symbol
L = 3;            % Channel order
CP_length = 4;    % Cyclic prefix length
B = 10;           % Number of OFDM symbols per transmitted frame
mc_N = 50;        % Number of iterations to achieve sufficient errors
th_var = 0:.1:1; % Clipping thresholds
Pe = zeros(size(th_var));       % Initializing the error vector
Total_length = (CP_length+N)*B; % Total length of each frame
am = [1,1i,-1,-1i];             % For QPSK
M = 4;                          % For QPSK

for th_loop = 1:length(th_var)
    th = th_var(th_loop);
    err = 0;

    for mc_loop = 1:mc_N
        dat_ind = ceil(M*rand(B,N));
        data = am(dat_ind);

        % Reshape the data into a BxN matrix,...
```

```
%...used later for error detection
data_reshape = reshape(data, 1, B*N);
tx_data = data;

for b = 1:B
    % Taking the IFFT
    data_t(b,:) = ifft(tx_data(b,:));
end

% Adding Cyclic prefix
data_cp = [data_t(:,end-CP_length+1:end), data_t];

% Reshaping the BxN matrix to...
%...obtain the frame (1xTotal_length)
data_tx =reshape(data_cp.',1,Total_length);

thu = abs(th);
thl = -abs(th);
data_clip = data_tx;
pt_high = find(data_clip>thu);
data_clip(pt_high) = thu;

pt_low = find(data_clip<thl);
data_clip(pt_low) = thl;
rec = data_clip;

% Reshape the recd signal...
%...into CP_length+N x B array
rec_reshaped = (reshape(rec, CP_length+N, B)).';

% Remove CP
rec_sans_cp = rec_reshaped(:,CP_length+1:end);

for bb = 1:B

    %Taking the FFT
    rec_f(bb,:) = fft(rec_sans_cp(bb,:));
end
```

```
        for b2 = 1:B
            % Extracting the OFDM symbol...
            %...from the "rec_f" matrix
            rec_symbol = transpose(rec_f(b2,:));

            % Calc the Euclidean dist assuming 1
            det1 = abs(rec_symbol-am(1)).^2;

            % Calc the Euclidean dist assuming +1i
            det2 = abs(rec_symbol-am(2)).^2;

            % Calc the Euclidean dist assuming -1
            det3 = abs(rec_symbol-am(3)).^2;

            % Calc the Euclidean dist assuming -1i
            det4 = abs(rec_symbol-am(4)).^2;

            % Concatenating the vectors
            det = [det1, det2, det3, det4];

            % Find symbol the recd signal is closest to
            [min_val, ind] = min(det, [], 2);

            % Generating the decoded symbols
            dec(b2,:) = am(ind);
        end
        % Reshape decoded symbols to calc error
        dec_reshape = reshape(dec, 1, B*N);

        % Compare dec_reshape against data_reshape to calculate errors
        err = err + sum(dec_reshape~=data_reshape);
    end
    % Calculate the probability of error
    Pe(th_loop) = err/(mc_N*B*N);
end

% Semilog plot of Pe vs. clipping threshold
semilogy(th_var,Pe)
```

Figure 6.7 shows the effect of clipping on the performance of an OFDM system. As expected, as the clipping threshold becomes smaller, the performance of the system deteriorates even though it is a noiseless, unit-gain frequency-flat channel. This is the trade-off between performance and power consumption. In order to keep the power consumption of the amplifier at the transmitter low, the clipping threshold is lowered, and this leads to poor performance. On the other hand, if the clipping threshold increases, the BER is lowered, but the power consumption increases.

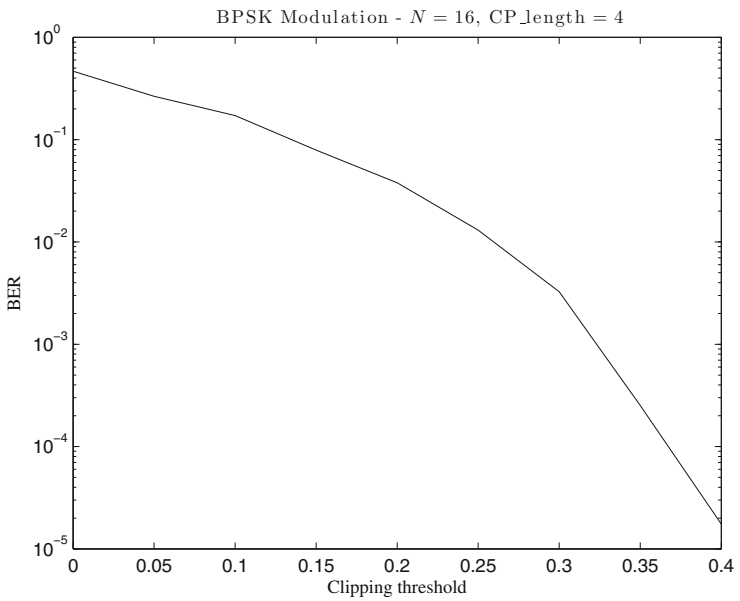


Figure 6.7: Effect of clipping on the BER of an OFDM system.